

Examen Parcial 1 Consultoría HPC con OpenMP

Consultoría Node Limit Conexiones que suman

Curso: CC3069 Computación Paralela y Distribuida

Semestre: II 2026

Link del Repo: <https://github.com/BARHUG0/ParalelaExamenParcial>

Integrantes

Javier Valladares, 23045

Hugo Barillas, 23556

Marinés García, 23391

1 Resumen del trabajo

En este trabajo usamos OpenMP para resolver dos problemas. El primero fue una Suma de Riemann, donde cada iteración tiene casi el mismo costo. El segundo fue la búsqueda de una ruta mínima con BFS. En ese caso el trabajo cambia según el tamaño de las fronteras y la cantidad de vecinos de cada nodo. Para medir el efecto del paralelismo comparamos cada versión con su programa secuencial.

La Suma de Riemann se paralelizó con reduction para evitar una condición de carrera en el acumulador. También usamos schedule(static), ya que las iteraciones son uniformes. En las pruebas de Javier Valladares, el mejor speedup fue de 5.624× con 8 hilos. El tiempo bajó de 0.968668 s a 0.172236 s.

Para BFS trabajamos por niveles. Las aristas se agruparon en bloques y se usó schedule(dynamic, 1) porque algunos nodos tienen muchos más vecinos que otros. Los resultados cambiaron bastante entre equipos. Hugo consiguió 3.658× con 8 hilos, mientras que Marínés obtuvo su mejor valor con 4 hilos, con un speedup de 1.142×.

2 Forma de realizar las pruebas

En cada equipo, las versiones secuencial y paralela se compilaron con el mismo nivel de optimización. La diferencia fue que la versión paralela agregó el soporte de OpenMP. Javier usó GCC 15.2.0 con -O3, C11 y -fopenmp para el programa paralelo. Hugo realizó sus pruebas en Windows con MinGW, usando un AMD Ryzen 5 8645HS. Marínés trabajó con un procesador Apple M5. En su evidencia de Riemann aparece Apple Clang 16.0.0 y un equipo con 10 CPU lógicas.

Las pruebas de Riemann de Javier y Marínés tuvieron una corrida de calentamiento y cinco repeticiones. En estos casos se reportó la mediana. El recolector de Hugo para Riemann y los recolectores del grafo hicieron tres repeticiones y calcularon el promedio. Por esta diferencia no usamos los tiempos absolutos para decir qué computadora es mejor. Cada speedup se obtuvo con la base secuencial del mismo equipo.

Las métricas usadas fueron:

$$\text{Speedup}(p) = T_{\text{secuencial}} / T_{\text{paralelo}}(p)$$

$$\text{Eficiencia}(p) = \text{Speedup}(p) / p * 100 \%$$

3 Problema 1 Suma de Riemann

3.1 Descripción y datos usados

Una suma de Riemann aproxima una integral definida al dividir el intervalo $[a,b]$ en n rectángulos con el mismo ancho:

$$\Delta x = (b-a)/n$$

$$x_i = a + i\Delta x$$

$$\text{área} \approx \sum f(x_i)\Delta x, i=0,\dots,n-1$$

La versión base usa los extremos izquierdos y acumula las áreas de forma secuencial. Su complejidad es $O(n)$ en tiempo y $O(1)$ en memoria. Cada punto se calcula con el índice, así que no hace falta guardar un arreglo completo.

Para las pruebas usamos $f(x)=x^2$ en $[0,1]$, con $n=10^9$. El valor exacto de la integral es $1/3$, lo que permite revisar el resultado. Con ese tamaño, la mayor parte del tiempo se dedica al ciclo de cálculo y no a crear los hilos. En las cinco repeticiones medidas, el error absoluto fue menor o igual a 10^{-8} .

3.2 Paralelización

```
#pragma omp parallel for default(none) shared(a, ancho, n, opcion) \
reduction(+: suma_areas) schedule(static)
```

Cada rectángulo puede calcularse por separado. Las variables a , ancho , n y opcion solo se leen y se comparten. Las variables i , x y y son privadas. El problema aparece en suma_areas , porque varias iteraciones podrían leerla y modificarla al mismo tiempo.

La cláusula `reduction` crea una suma parcial para cada hilo y las combina al final. De esta forma se evita la condición de carrera. Usar `critical` o `atomic` en cada rectángulo haría que una parte muy grande del ciclo se ejecutara de forma serial.

Elegimos `schedule(static)` porque todas las iteraciones calculan la misma función y cuestan casi lo mismo. OpenMP divide el trabajo una sola vez y cada hilo procesa cerca de n/p rectángulos.

3.3 Resultados de Javier Valladares

Las pruebas se hicieron en un Apple M1 Pro con 8 CPU lógicas y GCC 15.2.0.

Versión	Hilos	Mediana (s)	Speedup	Eficiencia
Secuencial	1	0.968668	1.000	100.00 %
OpenMP	1	0.964314	1.005	100.45 %
OpenMP	2	0.490168	1.976	98.81 %
OpenMP	4	0.250983	3.859	96.49 %
OpenMP	8	0.172236	5.624	70.30 %

Con 8 hilos, el tiempo bajó aproximadamente 82.22 %. La eficiencia con 4 hilos fue de 96.49 %, bastante cerca del comportamiento ideal. Al usar 8 hilos el programa siguió siendo más rápido, aunque la eficiencia cayó a 70.30 %. Esto puede relacionarse con el costo de la reducción, la sincronización, los recursos compartidos y los distintos tipos de núcleo del procesador.

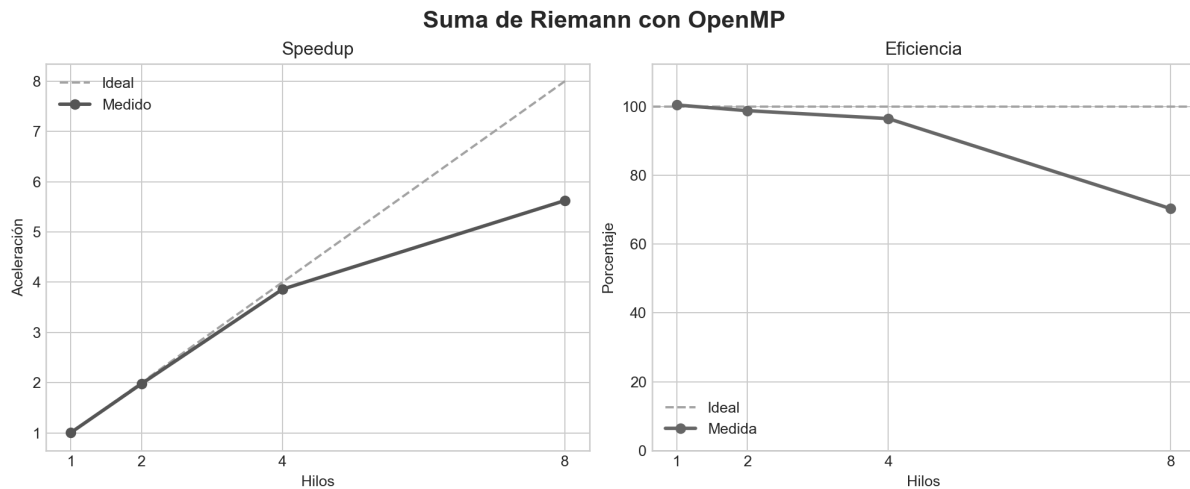


Figura 1. Speedup y eficiencia de Riemann

```

Evidencia real - benchmark OpenMP

$ make benchmark-riemann
Benchmark reproducible - Suma de Riemann
Fecha: 2026-09-09T20:14:29-06:00
Equipo: Apple M1 Pro (8 CPU lógicas)
Compilador: gcc-15 (Homebrew GCC 15.2.0_1) 15.2.0
Datos: f(x)=x^2, [0,1], n=1000000000, repeticiones=5

Modo      Hilos  Mediana(s)  Speedup  Eficiencia
-----
secuencial 1      0.968668    1.000    100.00%
openmp     1      0.964314    1.005    100.45%
openmp     2      0.490168    1.976    98.81%
openmp     4      0.250983    3.859    96.49%
openmp     8      0.172236    5.624    70.30%

Validación: todas las corridas tuvieron error absoluto <= 1e-08.
Archivos: docs/riemann/metricas_crudas.csv
          docs/riemann/metricas_resumen.csv

```

Figura 2. Evidencia de ejecución de Riemann

Los resultados completos están en `docs/riemann/metricas\_crudas.csv` y `docs/riemann/metricas\_resumen.csv`.

3.4 Resultados de Hugo Barillas

Hugo ejecutó tres repeticiones en Windows con MinGW, en una computadora con AMD Ryzen 5 8645HS. La tabla presenta el promedio de esas corridas.

Versión	Hilos	Promedio (s)	Speedup	Eficiencia
Secuencial	1	2.237247	1.000	100.00 %
OpenMP	1	2.420000	0.925	92.45 %
OpenMP	2	1.448667	1.544	77.22 %
OpenMP	4	0.763000	2.932	73.30 %
OpenMP	8	0.406667	5.501	68.77 %

El mejor resultado apareció con 8 hilos. El tiempo se redujo cerca de 81.82 %. OpenMP con un hilo fue más lento que el programa secuencial, pero a partir de dos hilos el costo adicional se compensó.

Los datos se encuentran en `resultados\_problema\_riemman\_hugo.csv` y `resumen\_problema\_riemman\_hugo.csv`.

3.5 Resultados de Marinés García

Marinés realizó las pruebas en un equipo con procesador Apple M5, 10 CPU lógicas y Apple Clang 16.0.0. Se hicieron cinco repeticiones con $n=10^9$.

Versión	Hilos	Mediana (s)	Speedup	Eficiencia
Secuencial	1	0.552750	1.000	100.00 %
OpenMP	1	0.538665	1.026	102.61 %
OpenMP	2	0.275918	2.003	100.17 %
OpenMP	4	0.141632	3.903	97.57 %
OpenMP	8	0.089596	6.169	77.12 %

Con 8 hilos, el tiempo disminuyó aproximadamente 83.79 %. Los valores de eficiencia mayores que 100 % con uno y dos hilos pueden aparecer por cambios de frecuencia, caché o pequeñas variaciones de medición. Por sí solos no prueban un comportamiento superlineal.

```

Evidencia real - benchmark OpenMP

$ make benchmark
Benchmark reproducible - Suma de Riemann
Fecha: 2026-09-09T20:13:17-06:00
Equipo: Apple M5 (10 CPU lógicas)
Compilador: Apple clang version 16.0.0 (clang-1600.0.26.6)
Datos: f(x)=x^2, [0,1], n=1000000000, repeticiones=5

Modo      Hilos  Mediana(s)  Speedup  Eficiencia
-----
secuencial 1      0.552750    1.000    100.00%
openmp      1      0.538665    1.026    102.61%
openmp      2      0.275918    2.003    100.17%
openmp      4      0.141632    3.903    97.57%
openmp      8      0.089596    6.169    77.12%

Validación: todas las corridas tuvieron error absoluto <= 1e-08.
Archivos: paralela/metricas_crudas.csv
          paralela/metricas_resumen.csv

```

Figura 3. Evidencia de Riemann de Marinés

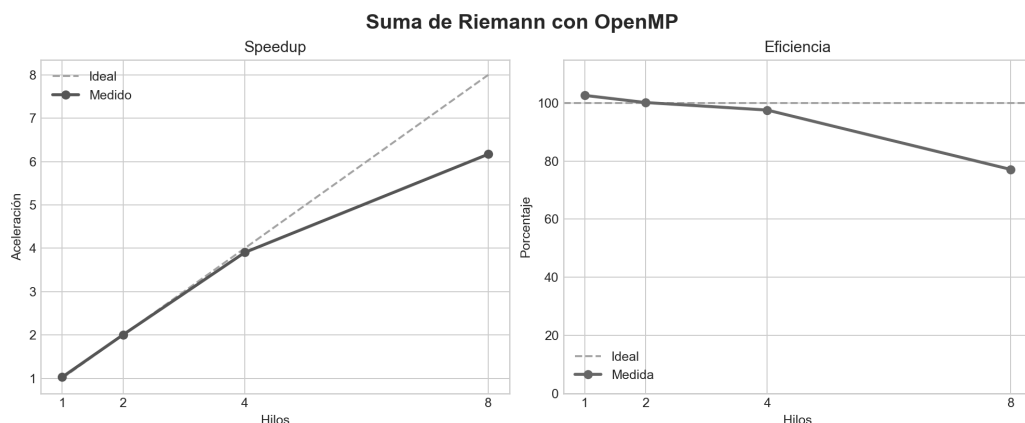


Figura 4. Speedup y eficiencia de Riemann de Marinés

La tabla también está guardada en `resumen\_problema\_riemann\_marines.csv`.

3.6 Comparación de resultados

Los tres equipos muestran un escalamiento parecido en Riemann. Con 8 hilos, los speedups fueron $5.624\times$ para Javier, $5.501\times$ para Hugo y $6.169\times$ para Marínés. Los tiempos cambian por el hardware, el compilador y las condiciones de cada ejecución. Aun así, en los tres casos `schedule(static)` y `reduction` funcionaron bien para este ciclo uniforme.

4 Problema 2 Ruta mínima en un grafo

4.1 Descripción y datos usados

El objetivo es encontrar el camino con menos aristas entre dos nodos de un grafo no ponderado. La solución base usa búsqueda en anchura, conocida como BFS. El algoritmo visita el grafo por niveles y guarda el padre de cada nodo para reconstruir la ruta.

El generador crea un grafo sintético y reproducible con semilla 12345. Tiene 1,000,000 de nodos, tres conexiones aleatorias base por nodo y 300 nodos hub. Cada hub recibe 10,000 conexiones adicionales. La prueba realiza 50 consultas entre pares conectados.

El grafo se guarda en formato CSR, con un arreglo de desplazamientos y otro de aristas. Así se evita una matriz de adyacencia de tamaño cuadrático y los vecinos de cada nodo quedan en posiciones contiguas de memoria.

4.2 Paralelización

El BFS paralelo mantiene el recorrido por niveles para conservar las rutas mínimas. Una región paralela procesa cada frontera. Dentro de ella, un bloque `single` divide las listas de vecinos en tareas de hasta 256 aristas. Después, un `omp for` con `schedule(dynamic, 1)` reparte esas tareas. Este reparto ayuda cuando los hubs tienen muchos más vecinos que los nodos comunes.

Usamos `atomic capture` para que un solo hilo reclame cada nodo no visitado y escriba su padre. Cada hilo guarda la siguiente frontera en un búfer privado. Luego, una sección crítica une esos búferes y las barreras separan los niveles. Con esto evitamos escrituras concurrentes sin volver serial todo el recorrido.

4.3 Resultados de Hugo Barillas

Estas pruebas se realizaron en Windows con MinGW y un AMD Ryzen 5 8645HS. Los CSV contienen tres repeticiones y la tabla muestra sus promedios.

Versión	Hilos	Promedio (s)	Speedup	Eficiencia
Secuencial	1	3.493667	1.000	100.00 %
OpenMP	1	3.431333	1.018	101.82 %
OpenMP	2	2.295000	1.522	76.11 %
OpenMP	4	1.442333	2.422	60.56 %
OpenMP	8	0.955000	3.658	45.73 %

El tiempo bajó cerca de 72.66 % con 8 hilos. La eficiencia fue menor que en Riemann, algo que coincide con el trabajo irregular de BFS y la sincronización que se hace en cada nivel. El valor mayor que 100 % con un hilo puede venir de la variación entre corridas o de diferencias de localidad. No representa paralelismo real.

Los datos se encuentran en ``resultados_problema_grafo_hugo.csv`` y ``resumen_problema_grafo_hugo.csv``.

4.4 Resultados de Marinés García

Marinés usó un equipo con procesador Apple M5. Los CSV incluyen tres repeticiones y los valores siguientes son promedios.

Versión	Hilos	Promedio (s)	Speedup	Eficiencia
Secuencial	1	0.930645	1.000	100.00 %
OpenMP	1	0.849841	1.095	109.51 %
OpenMP	2	0.865916	1.075	53.74 %
OpenMP	4	0.814876	1.142	28.55 %
OpenMP	8	0.905822	1.027	12.84 %

El mejor tiempo se obtuvo con 4 hilos. La reducción fue de aproximadamente 12.44 % y el speedup llegó a 1.142×. Con 8 hilos aumentó el costo de coordinación y el beneficio quedó en 1.027×. En esta prueba, usar más hilos no produjo una mejora mayor.

Los datos se encuentran en `resultados\_problema\_grafo\_marines.csv` y `resumen\_problema\_grafo\_marines.csv`.

4.5 Prueba local de Javier Valladares

También hicimos tres repeticiones en el Apple M1 Pro usado para Riemann. Estas corridas sirven como una prueba de integración y no reemplazan los CSV anteriores. Las 50 consultas tuvieron cero diferencias de longitud en cada repetición. Esto confirma que el BFS paralelo conservó el resultado de la versión secuencial.

Versión	Hilos	Promedio local (s)	Speedup	Eficiencia
Secuencial	1	2.197575	1.000	100.00 %
OpenMP	1	2.166460	1.014	101.44 %
OpenMP	2	3.031252	0.725	36.25 %
OpenMP	4	3.379260	0.650	16.26 %
OpenMP	8	2.978937	0.738	9.22 %

En este equipo, la versión paralela fue más lenta al usar entre 2 y 8 hilos. El costo de una operación atómica por arista, la sección crítica para unir las fronteras y las barreras de cada nivel superó el trabajo útil. Estos valores son un diagnóstico adicional y no sustituyen las mediciones de Hugo y Marinés.

5 Participación y evidencia

Integrante	Medición principal	Mejor configuración	Respaldo disponible
Javier Valladares	Riemann	8 hilos, 5.624×, 70.30 %	CSV, gráfica y evidencia de ejecución
Hugo Barillas	Grafo	8 hilos, 3.658×, 45.73 %	CSV crudo y resumen
Marinés García	Riemann	8 hilos, 6.169×, 77.12 %	CSV, gráfica y evidencia de ejecución

Hugo también midió Riemann y obtuvo 5.501× con 8 hilos. Marinés probó el grafo y consiguió 1.142× con 4 hilos. Recalculamos los promedios, speedups y eficiencias de los CSV recibidos. Los valores coinciden con los resúmenes guardados en el repositorio.

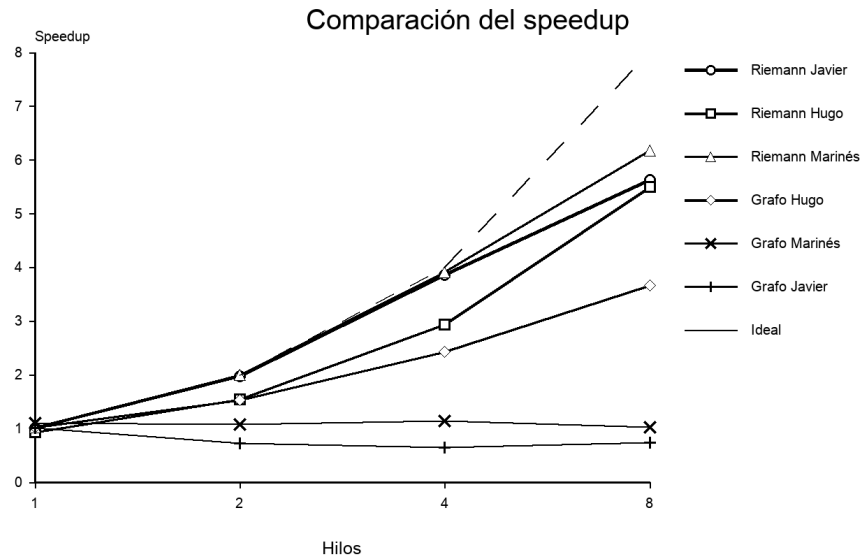


Figura 5. Comparación de speedup por integrante

6 Conclusiones

La Suma de Riemann tuvo un escalamiento fuerte en los tres equipos. Con 8 hilos se obtuvieron speedups entre $5.501\times$ y $6.169\times$. Esto tiene sentido porque las iteraciones son independientes, el reparto estático tiene poco costo y solo se necesita una reducción al final.

Los resultados de BFS dependieron más del equipo. Hugo llegó a $3.658\times$ con 8 hilos. Marínés obtuvo un máximo de $1.142\times$ con 4 hilos y la prueba de Javier no superó la base secuencial. Las operaciones atómicas, la sección crítica y las barreras por nivel pueden costar más que el trabajo útil si la carga o el procesador no compensan esa coordinación.

Las pruebas muestran que aumentar la cantidad de hilos no garantiza un programa más rápido. La estrategia y el tipo de schedule deben ajustarse al trabajo que hace el algoritmo. También se debe indicar el equipo, el compilador, la cantidad de repeticiones y la forma de resumir los tiempos para poder interpretar cada resultado.